Cycle-by-cycle pipeline traces (Sections 8.1-8.4)

Produced by simulating the shipped circuit with the test ROMs tests/t1_hazards.circ and tests/t2_branch.circ (all six test files are identical to washing_pipeline.circ apart from the ROM contents). Each row is the state during one clock cycle, i.e. after the previous rising edge has settled: the instruction held in each stage (PC:assembly; '-' = invalid entry with v=0), the valid bits of IF/ID, ID/EX, EX/MEM, MEM/WB, stall_eff, flush (= taken | halt_ex), the forwarding source of the instruction in EX (RF = register file, EX/MEM, MEM/WB), and the architectural writes that take effect at the edge ending the cycle. The debug output pins with the same names show these signals in Logisim.

8.1 ALU forwarding (no stall): ADD R1,R2 ; SUB R3,R1

Cyc	IF (PC)	ID	EX	MEM	WB	v ID EX MEM WB	stall	flush	fwd A	fwd B	RF write (WB)	store commit (MEM)
9	09:ADD R1, R2	08:NOP	07:NOP	06:NOP	05:ST R5, [R4]	1 1 1 1	0	0	RF	RF		
10	0A:SUB R3, R1	09:ADD R1, R2	08:NOP	07:NOP	06:NOP	1 1 1 1	0	0	RF	RF		
11	0B:NOP	0A:SUB R3, R1	09:ADD R1, R2	08:NOP	07:NOP	1 1 1 1	0	0	RF	RF		
12	0C:NOP	0B:NOP	0A:SUB R3, R1	09:ADD R1, R2	08:NOP	1 1 1 1	0	0	RF	EX/MEM		
13	0D:NOP	0C:NOP	0B:NOP	0A:SUB R3, R1	09:ADD R1, R2	1 1 1 1	0	0	RF	RF	R1<-15	
14	0E:LDI R2, 0x20	0D:NOP	0C:NOP	0B:NOP	0A:SUB R3, R1	1 1 1 1	0	0	RF	RF	R3<-1B	
15	0F:NOP	0E:LDI R2, 0x20	0D:NOP	0C:NOP	0B:NOP	1 1 1 1	0	0	RF	RF		

SUB R3,R1 in EX takes R1 (B-field) from EX/MEM (ADD, 0x15). No stall cycle, and R3 = 0x30 - 0x15 = 0x1B.

8.2 Load-use: LD R1,[R2] ; ADD R3,R1 (exactly one bubble)

Cyc	IF (PC)	ID	EX	MEM	WB	v ID EX MEM WB	stall	flush	fwd A	fwd B	RF write (WB)	store commit (MEM)
18	12:LD R1, [R2]	11:NOP	10:NOP	0F:NOP	0E:LDI R2, 0x20	1 1 1 1	0	0	RF	RF	R2<-20	
19	13:ADD R3, R1	12:LD R1, [R2]	11:NOP	10:NOP	0F:NOP	1 1 1 1	0	0	RF	RF		
20	14:NOP	13:ADD R3, R1	12:LD R1, [R2]	11:NOP	10:NOP	1 1 1 1	1	0	RF	RF		
21	14:NOP	13:ADD R3, R1	-	12:LD R1, [R2]	11:NOP	1 0 1 1	0	0	-	-		
22	15:NOP	14:NOP	13:ADD R3, R1	-	12:LD R1, [R2]	1 1 0 1	0	0	RF	MEM/WB	R1<-77	
23	16:NOP	15:NOP	14:NOP	13:ADD R3, R1	-	1 1 1 0	0	0	RF	RF		
24	17:LDI R1, 0x01	16:NOP	15:NOP	14:NOP	13:ADD R3, R1	1 1 1 1	0	0	RF	RF	R3<-92	
25	18:LDI R2, 0x02	17:LDI R1, 0x01	16:NOP	15:NOP	14:NOP	1 1 1 1	0	0	RF	RF		
26	19:LDI R3, 0x04	18:LDI R2, 0x02	17:LDI R1, 0x01	16:NOP	15:NOP	1 1 1 1	0	0	RF	RF		

Cycle with stall=1: the LD is in EX and ADD (reads R1) is in ID. PC (IF) and IF/ID hold, and ID/EX receives a bubble (v_EX=0 in the next row) while the LD moves on to MEM. Next cycle ADD enters EX while the LD is in MEM/WB, so fwd B = MEM/WB (0x77). R3 = 0x1B + 0x77 = 0x92 is written one cycle later than without the hazard. One stall cycle in total.

8.3 Youngest producer to a store: ADD R1,R2 ; SUB R1,R3 ; ST R1,[R4]

Cyc	IF (PC)	ID	EX	MEM	WB	v ID EX MEM WB	stall	flush	fwd A	fwd B	RF write (WB)	store commit (MEM)
31	1E:ADD R1, R2	1D:NOP	1C:NOP	1B:NOP	1A:LDI R4, 0x21	1 1 1 1	0	0	RF	RF	R4<-21	
32	1F:SUB R1, R3	1E:ADD R1, R2	1D:NOP	1C:NOP	1B:NOP	1 1 1 1	0	0	RF	RF		
33	20:ST R1, [R4]	1F:SUB R1, R3	1E:ADD R1, R2	1D:NOP	1C:NOP	1 1 1 1	0	0	RF	RF		
34	21:NOP	20:ST R1, [R4]	1F:SUB R1, R3	1E:ADD R1, R2	1D:NOP	1 1 1 1	0	0	EX/MEM	RF		
35	22:NOP	21:NOP	20:ST R1, [R4]	1F:SUB R1, R3	1E:ADD R1, R2	1 1 1 1	0	0	EX/MEM	RF	R1<-03	
36	23:NOP	22:NOP	21:NOP	20:ST R1, [R4]	1F:SUB R1, R3	1 1 1 1	0	0	RF	RF	R1<-FF	[21]<-FF
37	24:LDI R2, 0x20	23:NOP	22:NOP	21:NOP	20:ST R1, [R4]	1 1 1 1	0	0	RF	RF		
38	25:LDI R4, 0x22	24:LDI R2, 0x20	23:NOP	22:NOP	21:NOP	1 1 1 1	0	0	RF	RF		

With ST in EX both EX/MEM (SUB, R1=0xFF) and MEM/WB (ADD, R1=0x03) match the store data register R1. The youngest, EX/MEM, is chosen (fwd A = EX/MEM), and the address R4 = 0x21 comes from the RF. The store commits once, [21] <- FF, in MEM.

8.4 Taken branch with wrong-path actuator write

SUB R1,R1 ; BEQZ R1,TARGET ; ST R2,[R5] (R5 = 0xF4 = ACT, R2 = 0x01 = VALVE_IN) ; ADD R3,R4 ; TARGET: ...

Cyc	IF (PC)	ID	EX	MEM	WB	v ID EX MEM WB	stall	flush	fwd A	fwd B	RF write (WB)	store commit (MEM)
8	08:SUB R1, R1	07:NOP	06:NOP	05:NOP	04:LDI R1, 0x5A	1 1 1 1	0	0	RF	RF	R1<-5A	
9	09:BEQZ R1, target	08:SUB R1, R1	07:NOP	06:NOP	05:NOP	1 1 1 1	0	0	RF	RF		
10	0A:ST R2, [R5]	09:BEQZ R1, target	08:SUB R1, R1	07:NOP	06:NOP	1 1 1 1	0	0	RF	RF		
11	0B:ADD R3, R4	0A:ST R2, [R5]	09:BEQZ R1, target	08:SUB R1, R1	07:NOP	1 1 1 1	0	1	EX/MEM	RF		
12	0C:LDI R6, 0x40	-	-	09:BEQZ R1, target	08:SUB R1, R1	0 0 1 1	0	0	-	-	R1<-00	
13	0D:ST R3, [R6]	0C:LDI R6, 0x40	-	-	09:BEQZ R1, target	1 0 0 1	0	0	-	-		
14	0E:LDI R1, 0x07	0D:ST R3, [R6]	0C:LDI R6, 0x40	-	-	1 1 0 0	0	0	RF	RF		
15	0F:BNEZ R1, tgt2	0E:LDI R1, 0x07	0D:ST R3, [R6]	0C:LDI R6, 0x40	-	1 1 1 0	0	0	RF	EX/MEM		

BEQZ reaches EX with R1 forwarded from EX/MEM (SUB result 0). taken=1, so flush=1. At that edge PC <- TARGET (0x0C). IF/ID and ID/EX load v=0, which discards the ST that was in ID and the ADD that was being fetched: exactly two wrong-path instructions. Neither ever commits: no ST_COMMIT to 0xF4, VALVE_IN stays 0, and R3 keeps 0x11 (verified by storing R3 to RAM[0x40]).

Valid HALT: older store completes, younger instructions are killed (end of t2_branch)

Cyc	IF (PC)	ID	EX	MEM	WB	v ID EX MEM WB	stall	flush	fwd A	fwd B	RF write (WB)	store commit (MEM)
25	19:HALT	18:ST R6, [R7]	17:LDI R6, 0x42	16:LDI R7, 0x41	15:BEQZ R1, bad	1 1 1 1	0	0	RF	RF		
26	1A:LDI R6, 0xEE	19:HALT	18:ST R6, [R7]	17:LDI R6, 0x42	16:LDI R7, 0x41	1 1 1 1	0	0	EX/MEM	MEM/WB	R7<-41	
27	1B:ST R6, [R7]	1A:LDI R6, 0xEE	19:HALT	18:ST R6, [R7]	17:LDI R6, 0x42	1 1 1 1	0	1	RF	RF	R6<-42	[41]<-42
28	-	-	-	19:HALT	18:ST R6, [R7]	0 0 1 1	0	0	-	-		
29	-	-	-	-	19:HALT	0 0 0 1	0	0	-	-		
30	-	-	-	-	-	0 0 0 0	0	0	-	-		
31	-	-	-	-	-	0 0 0 0	0	0	-	-		

The two earlier wrong-path HALTs (after the taken BNEZ and JMP) were flushed in ID/IF and never asserted halt_ex. The final HALT reaches EX while the older ST R6,[R7] is in MEM: that store commits ([41] <- 42), and the younger LDI/ST are invalidated. From the next cycle IF shows '-' because HALTED=1 and fetching has stopped.